

EXPLANATION FOR QUESTION 1 -21

1. Meaning and Explanation to question 1

❖ pwd

The `pwd` command displays the current working directory. It tells me where I am located in the server's filesystem.

❖ whoami

The `whoami` command displays the username of the currently logged-in user. It helps me know what account I am using and what permissions I may have.

❖ uname -a

The `uname -a` command displays detailed information about the operating system and kernel, including the kernel version, system architecture, hostname, and operating system type.

2. Why does a DevSecOps engineer run these commands first when SSHing into an unknown server?

A DevSecOps engineer runs these commands first to establish situational awareness. Knowing the current directory prevents accidental changes in the wrong location. Knowing the logged-in user helps determine available privileges and access rights. Knowing the operating system and kernel version helps identify the platform being managed and allows security teams to check for known vulnerabilities and kernel CVEs that may affect the system. This information is essential for security assessments, troubleshooting, patch management, and safe administration of the server.

Q2 Answers

Five Directories and Their Purpose

1. /home

The /home directory stores personal files and folders for regular users. For example, the user "kali" has a home directory at /home/kali.

2. /etc

The /etc directory contains system-wide configuration files and settings used by the operating system and applications.

3. /var

The /var directory stores variable data such as logs, caches, mail, and application data that change frequently.

4. /usr

The /usr directory contains user programs, libraries, documentation, and other shared resources required by applications.

5. /tmp

The /tmp directory is used for temporary files created by users and applications. Files here may be deleted automatically by the system.

What is the Filesystem Hierarchy Standard (FHS)?

The Filesystem Hierarchy Standard (FHS) is a set of guidelines that defines the structure and organization of directories in Linux and Unix-like operating systems. It ensures that files and directories are stored in consistent locations across different Linux distributions.

Why Does FHS Exist?

FHS exists to provide consistency and predictability. System administrators, developers, and applications can easily locate files, configuration settings, logs, and programs regardless of the Linux distribution being used.

What Breaks on a Shared Production Server Without FHS?

Without FHS, systems would become difficult to manage because files could be stored in different locations on different servers. Scripts, applications, backups, monitoring tools, and automation processes might fail because they would not know where to find configuration files, logs, binaries, or user data. This would increase operational complexity, cause errors, and make troubleshooting much more difficult in production environments.

Q3 Answers

The difference between the two outputs?

The first command displays only the visible files and directories in the home directory. The second command (`ls -la`) displays both visible and hidden files and provides detailed information such as permissions, ownership, file size, and modification date.

Reason why a file is hidden in Linux?

A file is hidden in Linux when its name begins with a dot (`. `). It is not a special file type; it is simply a naming convention that causes most commands and file managers to hide it by default.

Two real hidden files found in a home directory and It explanation what they do.

❖ `.bashrc`

The `.bashrc` file stores shell configuration settings and commands that run whenever a new terminal session starts.

❖ `.profile`

The `.profile` file contains user-specific environment settings and startup commands that are executed when the user logs in.

Other examples include `.ssh`, which stores SSH keys and configurations, and `.config`, which stores application settings.

Q4 Answers

The flag I used and what it does ?

I used the `-p` flag with the `mkdir` command.

- The `-p` flag creates parent directories as needed and prevents errors if some directories already exist. It allows multiple nested directories to be created in a single command.

Meaning of brace expansion and how it help here?

Brace expansion is a Bash feature that generates multiple strings from a single pattern. It allows several directories to be specified compactly within curly braces `{ }`.

For example: `mkdir -p project/{docs,src,tests}`

creates the directories `docs`, `src`, and `tests` inside `project`.

In this task, brace expansion allowed me to create the entire directory structure, including nested subdirectories, in one command instead of creating each directory individually.

Could you have done this without it — and would you want to?

Yes, I could have created each directory separately using multiple ``mkdir`` commands. However, that would be slower, more repetitive, and more prone to mistakes. Using brace expansion makes the command shorter, faster, and easier to maintain, especially when creating complex directory structures.

Q5 Answers

Important of the `touch` command actually do beyond creating files?

The ``touch`` command creates empty files if they do not already exist. If a file already exists, it updates the file's access and modification timestamps instead of creating a new file.

What will have happened if I had run it on an existing file?

The file is not overwritten or deleted. The modification time and access time are updated to the current time.

Check `ls -l` before and after — what changes?

When comparing ``ls -l`` output before and after running ``touch`` on an existing file, the file's timestamp changes to the current date and time. The file size and contents remain unchanged.

Why does this behaviour matter in automation scripts?

This behaviour is useful because scripts can safely ensure required files exist without destroying existing data. Automation tools often use ``touch`` to create placeholder files, signal that a task has completed, or update timestamps for tracking and scheduling purposes.

Q6 Answers

What do the file sizes tell you about the files you just created?

The file sizes are 0 bytes, which means the files are empty. This is expected because they were created using the ``touch`` command as placeholder files.

Break down the permission string on one file.

Example: `-rw-r--r--`

Character breakdown:

➤ ``-`` = Regular file

- `rw-` = Owner permissions (read and write)
- `r--` = Group permissions (read only)
- `r--` = Other users permissions (read only)

This means the owner can read and modify the file, while group members and other users can only read it.

Essence of the -h flag add?

The `-h` flag displays file sizes in a human-readable format such as B (bytes), K (kilobytes), M (megabytes), and G (gigabytes), making the output easier to understand.

Q7 Answers

How is tree different from ls -R?

The `tree` command displays directories and files in a graphical tree-like structure, making it easy to understand the hierarchy.

The `ls -R` command lists files and directories recursively, but the output is shown as plain text and can be harder to read for large directory structures.

When would a DevSecOps engineer run tree on a live server?

A DevSecOps engineer may use `tree` to quickly inspect a project's directory structure, verify deployments, check whether required files and folders exist, and troubleshoot configuration issues.

They may also use it to identify unexpected files or directories, validate application layouts, and ensure automation scripts have created the correct structure.

Q8 Answers

What is the difference between > and >>?

The `>` operator writes output to a file and overwrites any existing contents.

The `>>` operator appends output to the end of a file without removing existing contents.

To prove it — what happened when i used > instead of >>?

When I re-ran the first echo command with `>`, the contents of the file were replaced. The previous log entries disappeared, and only the new line remained in the file.

Why is confusing these two operators dangerous in a production log rotation script?

Using `>` accidentally can erase important log data, making troubleshooting and auditing difficult. In a production environment, losing logs can hide errors, security events, or evidence needed for incident investigations. Using `>>` ensures new log entries are added while preserving existing records.

Q9 Answer

The difference between the two?

 `cat` displays the file from the first line to the last line.

 `tac` displays the file from the last line to the first line.

In a real incident with 80,000 log lines, which I would use first?

I would use `tac` first because the newest events are usually at the bottom of the log and are often the most relevant during troubleshooting.

The third command shows only the last 10 lines?

```
```bash
```

```
tail access.log
```

### Q10 Answer

Q10 Explains what is a heredoc and what it solve over multiple echo commands?

A heredoc allows multiple lines of text to be entered and written to a file in a single operation. It is cleaner and easier to manage than writing many separate echo commands.

What is the difference between EOF and 'EOF'?

With unquoted EOF, variables are expanded by the shell.

Example: Database: cyphercore

With quoted 'EOF', variables are treated as plain text.

Example: Database: \$DBNAME

Why might someone prefer a heredoc over nano or vim on a compromised server?

### **Q11 Answer**

Question 11 Explains The Key differences between more and less

`more` is a basic pager that allows forward navigation through a file.

`less` is more advanced and allows forward and backward navigation, searching, and jumping within a file.

Which loads the entire file into memory first?

`more` generally reads sequentially and is less flexible.

`less` is designed to efficiently handle very large files without loading the entire file into memory at once.

On a server with 512MB RAM and a 3GB log, which do you use?

I would use `less` because it is designed for viewing large files efficiently.

Three keyboard shortcuts for less

A heredoc allows quick file creation directly from the command line without opening an interactive editor.

Editors such as nano or vim may leave swap files, temporary files, backups, command histories, or editor-specific traces. A heredoc typically leaves fewer artefacts and is faster for simple changes.

### **Q12 ANSWER**

What is the fundamental difference between the two commands?

`cp` creates a copy of a file while leaving the original unchanged.

`mv` moves or renames a file. When used for renaming, the original filename is replaced by the new one.

After renaming, does the original filename still exist?

No. After using ``mv``, the original filename no longer exists. Only the new filename remains.

Does copying a large file duplicate all data on disk immediately?

Generally yes. A normal ``cp`` operation creates a separate copy of the file's data, consuming additional disk space. The time required depends on the file size and storage system.

### Q13 ANSWER

What happened when you tried to remove logs/?

The command failed because the ``logs`` directory contains files and subdirectories. ``rmdir`` can only remove empty directories.

Difference between `rmdir` and `rm -r`

 ``rmdir`` removes only empty directories.

 ``rm -r`` removes directories and everything inside them recursively.

What makes `rm -rf` dangerous?

``rm -rf`` deletes files and directories recursively without asking for confirmation. A mistake in the path can delete important data very quickly.

What rule should every engineer follow before running it?

Always double-check the path and verify the target directory before pressing Enter. Many engineers run ``pwd`` and ``ls`` first to confirm they are in the correct location.

### Q14 ANSWER

The directory structure follows the intended project layout, with separate locations for configuration files, logs, archived logs, and reports.

Automation scripts depend on files being in expected locations. Log collection agents need to know where logs are stored. Security monitoring tools rely on consistent paths to collect, analyze, and alert on events.

A predictable structure reduces errors, simplifies troubleshooting, improves maintainability, and makes automation more reliable across environments.

### **Q15 ANSWER**

Both db.conf and db\_hardlink.conf have the same inode number. This shows they point to the same underlying file data.

The link count shows how many filenames point to the same inode.

An inode is a data structure that stores information about a file such as ownership, permissions, timestamps, and the locations of its data blocks on disk. Filenames point to inodes.

### **Q16 ANSWER**

The data remains accessible through db\_hardlink.conf.

The inode still exists because at least one filename still points to it. The link count decreased, but it did not reach zero.

All hard links pointing to the inode must be removed and no process can still have the file open.

The link count decreased by one after deleting db.conf.

An attacker could create additional hard links to important files so data remains accessible even after one filename is removed, potentially complicating forensic investigations.

### **Q17 ANSWER**

The letter `l` at the beginning of the permissions string.

The command failed because the target file no longer exists.

A dangling symlink or broken symlink.

Why are dangling symlinks difficult in deployments?

Applications and scripts may fail unexpectedly because the link exists but points to a missing file, making troubleshooting more difficult.

### Q18 ANSWER

PROPERTY	HARD LINK	SOFT LINK
SHARES INODE WITH ORIGINAL?	YES	NO
WORKS ACROSS DIFFERENT FILESYSTEMS?	NO	YES
SURVIVES DELETION OF ORIGINAL?	YES	NO (BECOMES BROKEN)
CAN LINK TO A DIRECTORY?	NO (GENERALLY RESTRICTED)	YES
SHOWS AS L IN LS -LA?	NO	YES
DETECTABLE BY MATCHING INODES IN LS -LI?	YES	NO

Hard links can be used to keep access to important data even if one filename is removed, such as preserving log files during maintenance operations.

Symbolic links are commonly used to point applications to the current configuration, release directory, or shared resource without changing application code.

### Q19ANSWER

The normal output from the existing directory was saved into `stdout\_output.txt` because standard output uses stream 1.

The error message from the missing directory was saved into ``stderr_output.txt`` because error messages use stream 2.

- `stdin (0)`: Carries input given to a command.
- `stdout (1)`: Carries normal command output.
- `stderr (2)`: Carries error messages and warnings.

``2>`` redirects only the error stream (`stderr`) into a file. It does not affect normal output.

``2>&1`` combines `stderr` with `stdout` by sending error messages to the same place as normal output.

A DevSecOps engineer may use it in deployment scripts or monitoring commands to capture both successful output and errors in one log file.

## Q20 ANSWER

I used an unquoted delimiter (``EOF``) because I wanted variables to be expanded inside the heredoc if needed.

Difference between quoted and unquoted heredoc

An unquoted heredoc: ``<< EOF``

allows variable expansion. Example:

If ``ANALYST="Abigail"``` is set, ``$ANALYST`` becomes ``Abigail``.

A quoted heredoc: ``<< 'EOF'``

does not expand variables. It writes the text exactly as it appears.

Example: ``$ANALYST`` remains ``$ANALYST``.

Use an unquoted heredoc when creating files that need dynamic values, such as deployment configuration files containing environment variables.

Use a quoted heredoc when writing exact text where variables should not change, such as scripts, templates, or documentation.

## Q21 ANSWER

This lab reflects real DevSecOps work. Directory structures help automation tools locate files. Logs help engineers investigate incidents. Archives preserve historical records. Links simplify deployments and configuration management. Reports document findings and operational status.

Two commands that changed how I think about Linux were:

1. ``ln`` - It showed that filenames are separate from the underlying inode and data.
2. ``tree`` - It made it easier to visualize large directory structures and verify deployments quickly.

Both commands are useful beyond this bootcamp because they help with troubleshooting, system administration, and production operations.